

Assignment 1 Report: $5 \times 5 \times 5$ Interactive 3D Cubic Grid

Name: T. Gnana Sasi Kiran

Roll No: 112301033

[github](#)

Name: R. Likhith Chandra

Roll No: 112301028

1. Core Working Principle

The program is built on **state-based 3D rendering**. It keeps the current state of a $5 \times 5 \times 5$ grid and of a separate movable $1 \times 1 \times 1$ cube. Keyboard input only *modifies this state*. In every frame the state is converted into OpenGL vertex geometry, uploaded to the GPU through a Vertex Buffer Object (VBO), transformed with Model-View-Projection matrices, and rasterized by OpenGL onto the screen.

note:- Added a colorful 3D helix model to make the scene more visually appealing and demonstrate additional 3D line rendering using OpenGL.

Keyboard input $\rightarrow$ Update scene state $\rightarrow$ Generate geometry $\rightarrow$ Upload to GPU $\rightarrow$ Vertex
shader $\rightarrow$ Rasterization $\rightarrow$ Screen

2. Scene State

- **Grid:** Cell `cells[5][5][5]` holds 125 unit cells; each stores **filled** (permanent cube or not) and its RGB color.
- **Movable cube:** `cube_x`, `cube_y`, `cube_z` (logical position) and `cube_color`.
- **Rotation:** `rotx`, `roty` rotate the whole model; they never change the logical position.

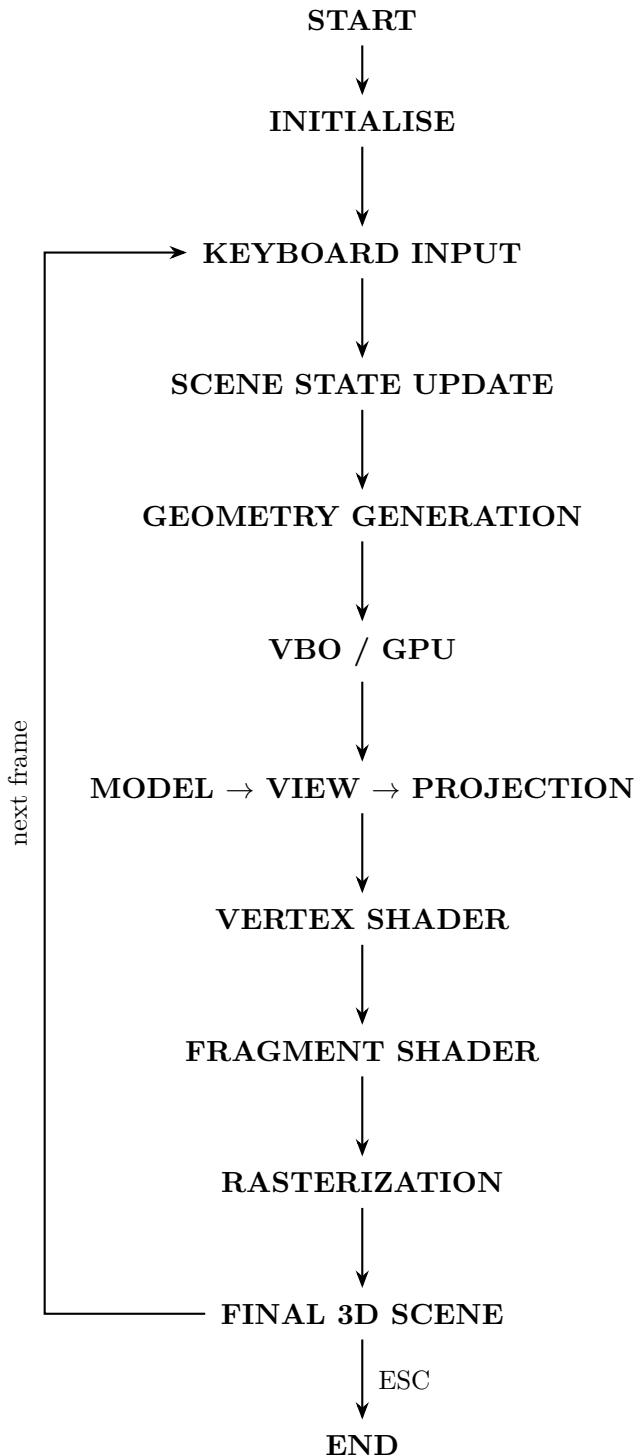
The cube is stored separately from `cells[][][]`, so it can move away after **F** is pressed while the filled cell stays stored.

3. Keyboard Controls

Key	Action	State change
LEFT / RIGHT	<code>moveCube($\mp 1, 0, 0$)</code>	$X - 1 / X + 1$
UP / DOWN	<code>moveCube($0, \pm 1, 0$)</code>	$Y + 1 / Y - 1$
U / B	<code>moveCube($0, 0, \pm 1$)</code>	$Z + 1 / Z - 1$
C	change cube colour (RGB from terminal, each in $[0, 1]$)	<code>cube_color</code> updated
F	fill current cell	<code>filled = true</code> , store <code>cube_color</code>
W	clear current cell	<code>filled = false</code>
L / R / T / D	rotate scene	<code>rotx</code> , <code>roty</code> change
ESC	close window	exit render loop

Boundary check: `moveCube()` accepts a new position (n_x, n_y, n_z) only if $0 \leq n_x, n_y, n_z < 5$, so the cube can never leave the grid.

4. Program Flow Chart



GLFW window and OpenGL context, GLEW, keyboard callback, compile/link shaders, VAO/VBO, initial `cells[5][5][5]` and cube state.

GLFW calls `keyCallback()` (`window.cpp`). Keys only *change state*: arrows/U/B move, C colour, F fill, W clear, L/R/T/D rotate.

`moveCube()` accepts a move only if $0 \leq n < 5$. Updates `cube_x/y/z`, `cube_color`, `cells[x][y][z]`, `rotx/roty`.

`createGrid()` $\rightarrow$ lines; `addCube()` for each filled cell and the selected cube (6 faces $\times$ 2 triangles $\times$ 3 = 36 vertices); plus the 3D line model. Vertex = (X, Y, Z, R, G, B) .

`drawData()`: bind VAO/VBO, `glBufferData()`, then `glDrawArrays()` with `GL_LINES` (grid, line model) or `GL_TRIANGLES` (cubes).

$\text{transform} = \text{projection} \times \text{view} \times \text{model}$, sent by `glUniformMatrix4fv()`. One model matrix $\Rightarrow$ everything rotates together.

`gl_Position = transform * vec4(pos, 1.0);`
colour is passed on through `ourcolor`.

`fragcolor = vec4(ourcolor, 1.0)` – the stored vertex colour becomes the visible colour.

OpenGL converts the transformed primitives into pixels.

`glfwSwapBuffers()` then `glfwPollEvents()`, so new key presses update the state for the next frame.

5. Output Screenshots

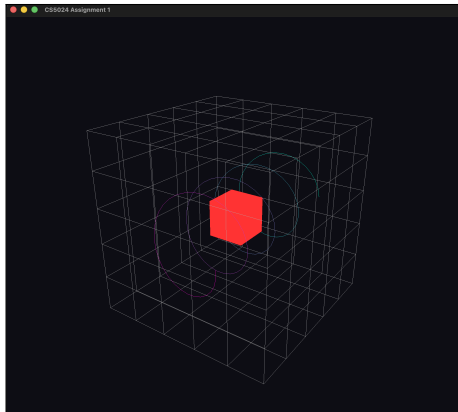


Fig. 1: Initial $5 \times 5 \times 5$ grid with the movable cube.

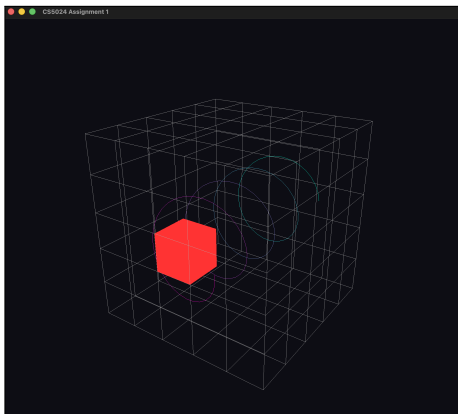


Fig. 2: Cube moved with arrow keys / U / B.

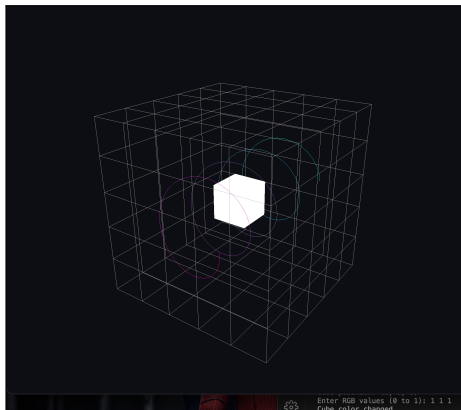


Fig. 3: Cube colour changed with C.

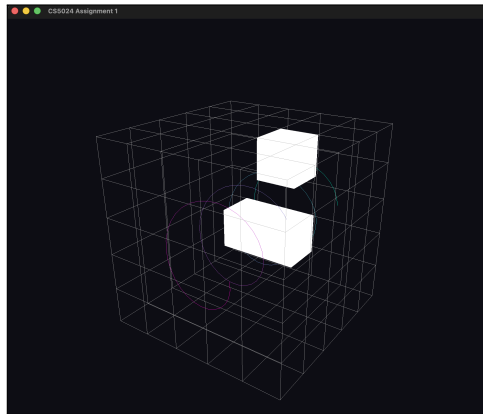


Fig. 4: Cells filled with **F** persist after moving.

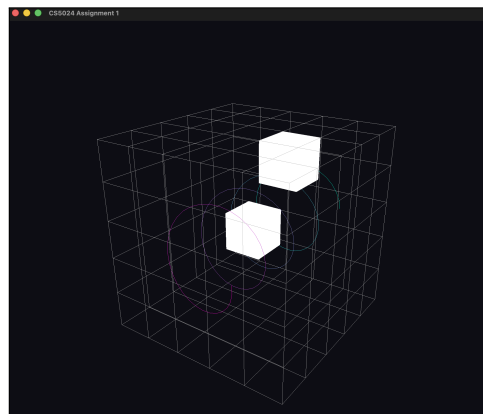


Fig. 5: A filled cell cleared with **W**.

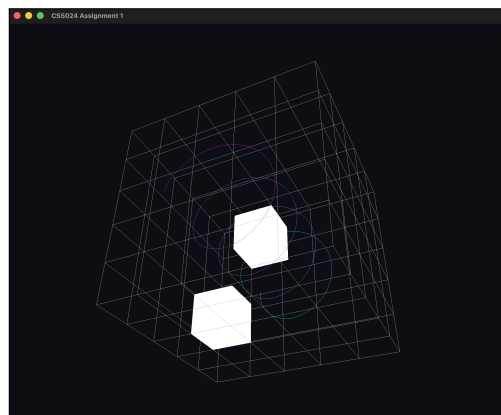


Fig. 6: Model rotated (L/R/T/D); everything rotates together.

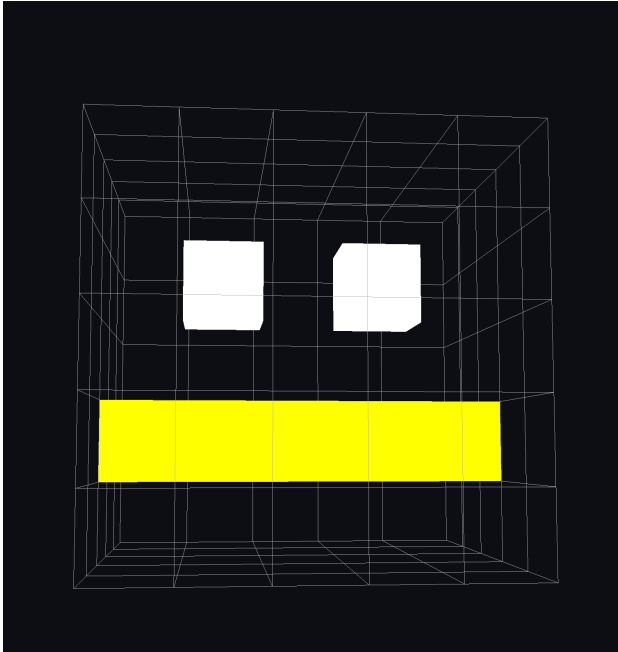
6. Implementation Details

<code>main.cpp</code>	Scene state, geometry, MVP, rendering
<code>window.cpp</code>	Window/context, <code>keyCallback()</code>
<code>shader.cpp</code>	Compile and link GLSL shaders
<code>vertex.glsl</code>	Transforms position, passes colour
<code>fragment.glsl</code>	Outputs final colour
<code>CMakeLists.txt</code>	Build system

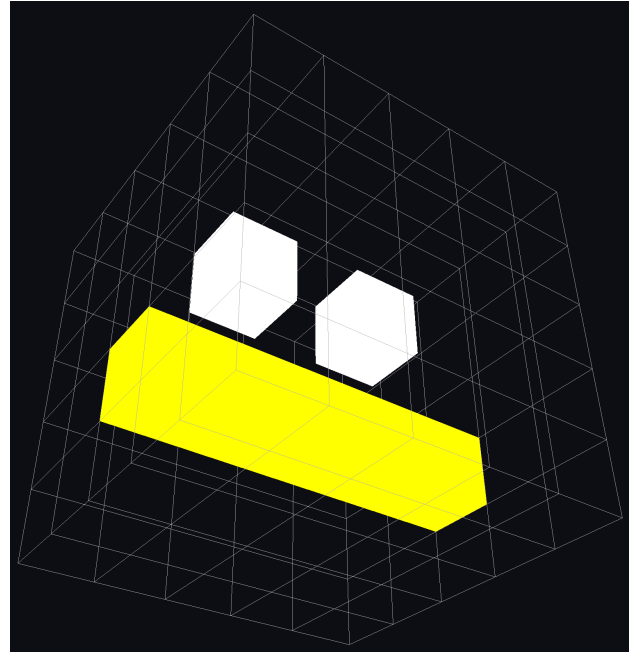
- C++ with OpenGL, GLFW, GLEW and GLSL; built with CMake.
- Geometry is rebuilt every frame from the state.
- Grid and line model use `GL_LINES`; cubes use `GL_TRIANGLES`.
- Vertex format: (X, Y, Z, R, G, B) ; same MVP for all objects.

7. A Model of an Emoji

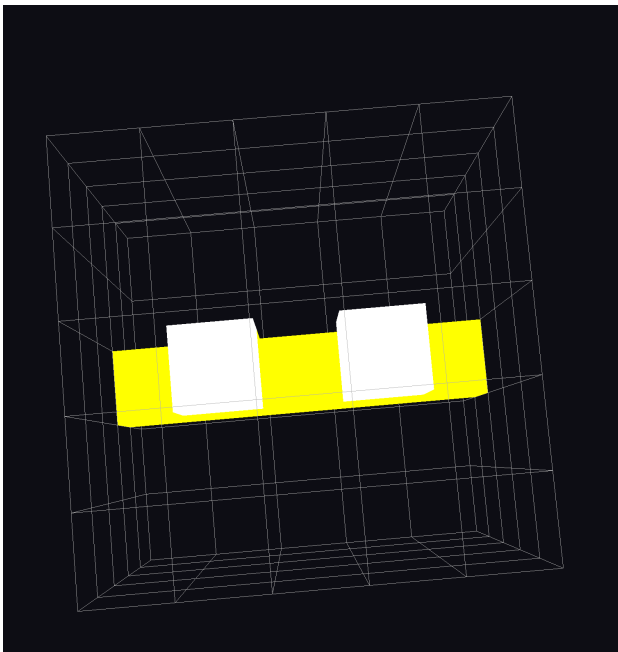
Using the same interactive controls, several cells of the $5 \times 5 \times 5$ grid were filled with **F** to build a simple **emoji face**: two white eye blocks in the upper half and a long yellow mouth block across the lower half. The model was then rotated with **L** / **R** / **T** / **D** to inspect it from different directions. Since rotation only changes **rotx** and **roty** and never the logical cell positions, the stored cells stay exactly where they were filled — only the camera-facing orientation of the whole scene changes.



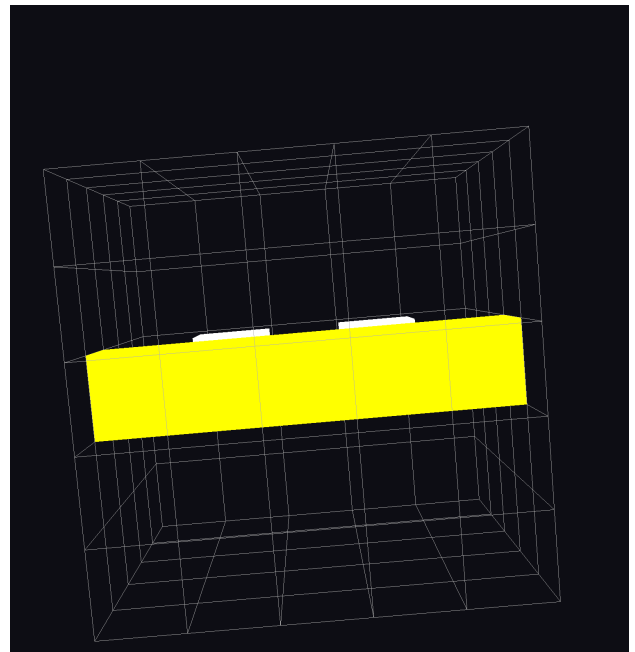
(a) Front view — the emoji face as seen head-on (no rotation applied).



(b) Isometric (corner) view — rotated with both L/R and T/D.



(c) Top view — rotated with T; the top faces of the eye and mouth blocks become visible.



(d) Bottom view — rotated with D; the mouth block occludes the eyes from this angle.

Figures 7–10: The emoji model viewed from four different orientations.